Practical 1

Part A — Practice Programs (Hour 1)

Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home.

1. Vending machine (VendingMachine.java, main in this file)

(a) Define an enum Coin with constants ONE, TWO, FIVE, TEN.

(b) In main, set a snack price of 15 and a running total of 0; create a Scanner.

(c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far.

(d) Stop the loop once the total reaches 15 or more.

(e) Print the change to return (total − 15).

Expected output: TEN then FIVE prints “Paid. Change: 0”; TEN then TEN prints “Paid. Change: 5”.

Sol)

Vendingmachine.java

```
import java.util.Scanner;


public class VendingMachine {
   // (a) Enum for accepted coins
   enum Coin {
      ONE, TWO, FIVE, TEN
   }


   public static void main(String[] args) {
      // (b) Set target price and total
      final int PRICE = 15;
      int total = 0;
      Scanner scanner = new Scanner(System.in);
```

```
System.out.println("Snack price: 15. Enter coins (ONE, TWO, FIVE, TEN):");

// (c) & (d) Loop until total reaches 15 or more
while (total < PRICE) {
   System.out.print("Insert coin: ");
   String input = scanner.next().toUpperCase();

   try {
      Coin coin = Coin.valueOf(input);

      // Switch expression to derive coin value
      int value = switch (coin) {
         case ONE -> 1;
         case TWO -> 2;
         case FIVE -> 5;
         case TEN -> 10;
      };

      total += value;
      System.out.println("Total inserted so far: " + total);

   } catch (IllegalArgumentException e) {
      System.out.println("Invalid coin! Please enter ONE, TWO, FIVE, or TEN.");
   }
}

// (e) Calculate and print change
int change = total - PRICE;
System.out.println("Paid. Change: " + change);
```

```
        scanner.close();
    }
}
```

Expected Output

Snack price: 15. Enter coins (ONE, TWO, FIVE, TEN):

Insert coin: TEN

Total inserted so far: 10

Insert coin: TEN

Total inserted so far: 20

Paid. Change: 5

Learning Outcome

Enums & Input Parsing: Learning to define strong types using Java `enum` constants and converting string inputs safely via `Enum.valueOf()`

Switch Expressions: Utilizing modern Java `switch` expressions to return values directly into variables without `break` statements.

Accumulator Loop Control: Managing running totals within a loop that terminates based on a quantitative condition ($\text{total} \ge 15$).

2. Toll booth (TollBooth.java, main in this file)

(a) Define a record Vehicle(String number, String type).

(b) In main, keep a running toll total and three counters (bike, car, truck).

(c) Loop until the user types “done” for the number: build a Vehicle, use a switch

expression on its type for the toll (bike→20, car→50, truck→150), add to the total,

and increment that type’s counter.

(d) After the loop, print the total toll and the type with the highest count.

Expected output: two cars + one bike → “Total toll: 120”, “Most frequent: car”.

Sol)

ToolBoth.java

```
import java.util.Scanner;


public class TollBooth {
```

```
    // (a) Vehicle record definition
    record Vehicle(String number, String type) {}

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int totalToll = 0;
        int bikeCount = 0;
        int carCount = 0;
        int truckCount = 0;

        // (c) Loop until user enters "done"
        while (true) {
            System.out.print("Enter vehicle number (or 'done' to stop): ");
            String number = scanner.next();

            if (number.equalsIgnoreCase("done")) {
                break;
            }

            System.out.print("Enter vehicle type (bike, car, truck): ");
            String type = scanner.next().toLowerCase();

            Vehicle vehicle = new Vehicle(number, type);

            // Switch expression to derive toll and track count
            int toll = switch (vehicle.type()) {
                case "bike" -> {
                    bikeCount++;
                    yield 20;
                }
```

```
            case "car" -> {
                carCount++;
                yield 50;
            }
            case "truck" -> {
                truckCount++;
                yield 150;
            }
            default -> {
                System.out.println("Unknown vehicle type. No toll added.");
                yield 0;
            }
        };

        totalToll += toll;
    }

    // (d) Determine vehicle type with highest count
    String mostFrequent = "none";
    int maxCount = Math.max(bikeCount, Math.max(carCount, truckCount));

    if (maxCount > 0) {
        if (maxCount == carCount) {
            mostFrequent = "car";
        } else if (maxCount == bikeCount) {
            mostFrequent = "bike";
        } else {
            mostFrequent = "truck";
        }
    }
```

```
        System.out.println("Total toll: " + totalToll);
        System.out.println("Most frequent: " + mostFrequent);


        scanner.close();
    }
}
```

Expected Output

```
Enter vehicle number (or 'done' to stop): AB123
Enter vehicle type (bike, car, truck): car
Enter vehicle number (or 'done' to stop): CD456
Enter vehicle type (bike, car, truck): car
Enter vehicle number (or 'done' to stop): EF789
Enter vehicle type (bike, car, truck): bike
Enter vehicle number (or 'done' to stop): done
Total toll: 120
Most frequent: car
```

Learning Outcome

Java Records: Understanding immutable, concise data modeling using Java `record` declarations.

Switch Block `yield` Keyword: Mastering block-style switch expressions that execute multi-line logic and return values using `yield`

Data Summarization: Aggregating multiple counters and computing comparative metrics (maximum occurrence) across dynamic input data.

3. (For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java)

(a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK.

(b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases).

(c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call

winner(), print the round result and keep score.

(d) After 5 rounds, print the overall winner.

Expected output: each round prints both moves and the round winner; ends with a "You win 3–2" summary.

Sol)

RPSLS.java

```
import java.util.Random;
import java.util.Scanner;

public class RPSLS {
   // (a) Enum for move options
   enum Move {
      ROCK, PAPER, SCISSORS, LIZARD, SPOCK
   }

   // (b) Returns 1 if 'a' wins, -1 if 'b' wins, 0 for tie
   public static int winner(Move a, Move b) {
      if (a == b) return 0;

      boolean aWins = switch (a) {
         case SCISSORS -> (b == Move.PAPER || b == Move.LIZARD);
         case PAPER    -> (b == Move.ROCK || b == Move.SPOCK);
         case ROCK     -> (b == Move.LIZARD || b == Move.SCISSORS);
         case LIZARD   -> (b == Move.SPOCK || b == Move.PAPER);
         case SPOCK    -> (b == Move.SCISSORS || b == Move.ROCK);
      };

      return aWins ? 1 : -1;
   }

   public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
        Random random = new Random();
        Move[] moves = Move.values();

        int playerScore = 0;
        int computerScore = 0;

        System.out.println("=== Rock-Paper-Scissors-Lizard-Spock ===");

        // (c) Play 5 rounds
        for (int round = 1; round <= 5; round++) {
            System.out.println("\n--- Round " + round + " ---");

            // Computer picks random move
            Move computerMove = moves[random.nextInt(moves.length)];

            System.out.print("Enter your move (ROCK, PAPER, SCISSORS, LIZARD, SPOCK): ");
            String input = scanner.next().toUpperCase();

            Move playerMove;
            try {
                playerMove = Move.valueOf(input);
            } catch (IllegalArgumentException e) {
                System.out.println("Invalid move! You forfeit this round.");
                computerScore++;
                continue;
            }

            System.out.println("You played: " + playerMove);
            System.out.println("Computer played: " + computerMove);
```

```
            int result = winner(playerMove, computerMove);

            if (result == 1) {
                System.out.println("Outcome: You win this round!");
                playerScore++;
            } else if (result == -1) {
                System.out.println("Outcome: Computer wins this round!");
                computerScore++;
            } else {
                System.out.println("Outcome: It's a tie!");
            }
        }

        // (d) Print overall result
        System.out.println("\n==================================");
        System.out.println("Final Result: You " + (playerScore >= computerScore ? "win" : "lose") +
                        " " + playerScore + "–" + computerScore);
        System.out.println("==================================");

        scanner.close();
    }
}
```

Expected Output

```
=== Rock-Paper-Scissors-Lizard-Spock ===

--- Round 1 ---
Enter your move (ROCK, PAPER, SCISSORS, LIZARD, SPOCK): SPOCK
You played: SPOCK
Computer played: ROCK
Outcome: You win this round!
```

--- Round 2 ---

Enter your move (ROCK, PAPER, SCISSORS, LIZARD, SPOCK): LIZARD

You played: LIZARD

Computer played: SCISSORS

Outcome: Computer wins this round!

--- Round 3 ---

Enter your move (ROCK, PAPER, SCISSORS, LIZARD, SPOCK): PAPER

You played: PAPER

Computer played: SPOCK

Outcome: You win this round!

--- Round 4 ---

Enter your move (ROCK, PAPER, SCISSORS, LIZARD, SPOCK): ROCK

You played: ROCK

Computer played: ROCK

Outcome: It's a tie!

--- Round 5 ---

Enter your move (ROCK, PAPER, SCISSORS, LIZARD, SPOCK): SPOCK

You played: SPOCK

Computer played: SCISSORS

Outcome: You win this round!

==========

Final Result: You win 3–1

Larning Outcome

Combinatorial Game Rules in Switch Statements: Modeling intricate game relationships (5 options with 2 winning conditions each) using concise boolean expressions within switch arms.

Randomized Game Logic: Using array indexing driven by `java.util.Random` to simulate unpredictable computer actions.

Multi-Round Game State Management: Structuring round-based loops while tracking scores and handling invalid inputs gracefully.